R3.01

Servelt

Au premier CM

Applications Web :

- reposent sur les architectures n-tiers
- les fonctionnalités sont réparties sur plusieurs serveurs communicants

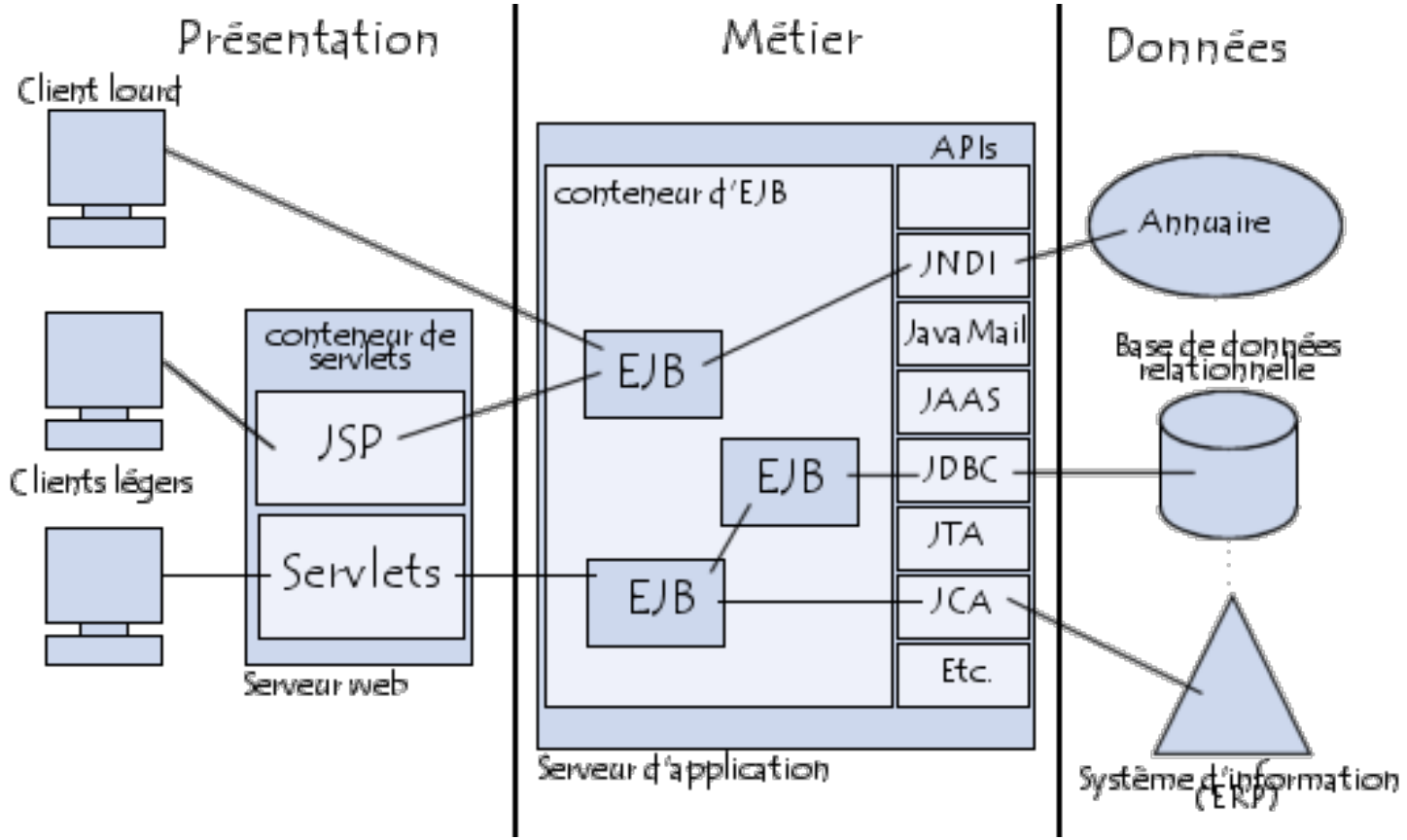
3 couches :

Présentation

Traitement (métiers)

Accès aux données

Architecture JEE



https://web.maths.unsw.edu.au/~lafaye

Servlet

Un ou une servlet est une classe Java qui permet de créer dynamiquement des données au sein d'un serveur HTTP. Ces données sont le plus généralement présentées au format HTML, mais elles peuvent également l'être au format XML, JSON ou tout autre format destiné aux navigateurs web. Les servlets utilisent l'API Java Servlet (package javax.servlet).

Les servlets d’exécutent dans un moteur de servlet.

Moteur de Servlet

Le moteur de servlet :

- gérer le cycle de vie : Il charge, instancie et initialise la servlet, puis appelle ses méthodes de traitement (doGet, doPost, etc.) et finit par la détruire.
- Fait l'interface réseau : Il fait le pont entre le serveur web et le code Java, évitant au développeur de gérer manuellement les sockets réseau ou le formatage des requêtes/réponses HTTP.
- Permet la concurrence : Il traite chaque requête entrante via des threads séparés pour répondre à plusieurs clients en même temps.

Le context

Un serveur web héberge plusieurs applications

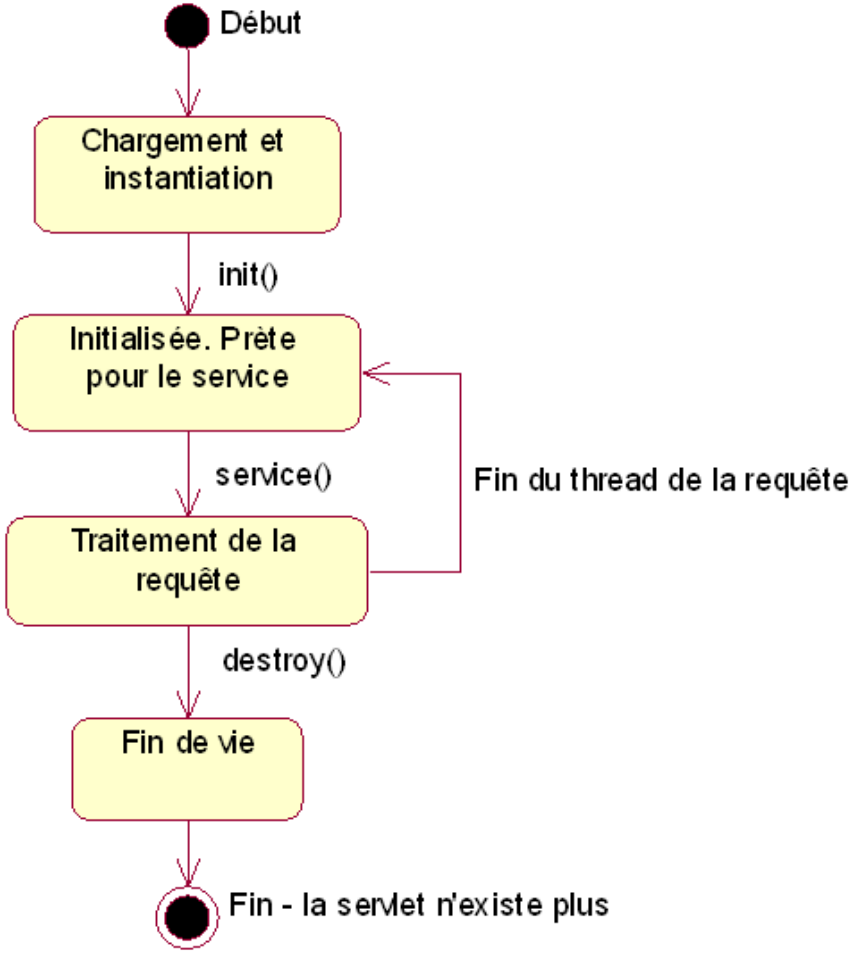
dans webapps pour tomcat

L’application est choisie en fonction de l’URL

http://localhost:8888/demo (demo est une application)

Chaque application a son propre context qui suit le nom d’hôte

Cycle de vie



http://remy-manu.no-ip.biz/Java/Tutoriels/J2EE/servlet.htm

1. Chargement et Instanciation (au démarrage ou à la première consultation)
   - Le conteneur charge la classe de la servlet en mémoire.
   - Il crée **une unique instance** de cette servlet.
2. Initialisation
   - La méthode init(ServletConfig config) est appelée **une seule fois** juste après l'instanciation.
3. Traitement des requêtes
   - À chaque fois qu'un client envoie une requête, le conteneur appelle la méthode service(ServletRequest req, ServletResponse res).
   - La méthode service() analyse le type de requête HTTP (GET, POST, PUT, DELETE...) et la disloque vers la méthode correspondante (doGet(), doPost(), etc.).
4. Destruction
   - Lorsque l'application web est arrêtée ou que la servlet est déchargée du serveur, la méthode destroy() est appelée une seule fois.

Mapping URL Servlet

Première approche : web.xml dans WEB-INF/

```
<web-app xmlns="https://jakarta.ee" version="6.0">
<!-- 1. Déclaration du Servlet -->
          <servlet>
                    <servlet-name>IdentifiantUnique</servlet-name>
                    <servlet-class>com.exemple.MonServlet</servlet-class>
          </servlet>
<!-- 2. Liaison de l'URL au Servlet déclaré -->
          <servlet-mapping>
                    <servlet-name>IdentifiantUnique</servlet-name>
                    <url-pattern>/mon-adresse</url-pattern>
          </servlet-mapping>
</web-app>
```

Mapping URL Servlet

Deuxième approche : l'annotation @WebServlet dans la classe

```
@WebServlet(
name = "BankAccountServlet",
description = "Represents a Bank Account and it's
transactions",
urlPatterns = {"/account", "/bankAccount" },
initParams = {
    @WebInitParam(name = "type", value =
"savings")})
```

Mapping URL Servlet

Deuxième approche : l’annotation @WebServlet dans la classe

- *@WebServlet : a*ssocier un motif d'URL spécifique (URL pattern) à une servlet
- *@WebInitParam :* pour **déclarer un paramètre d'initialisation** destiné à une Servlet ou à un Filtre.

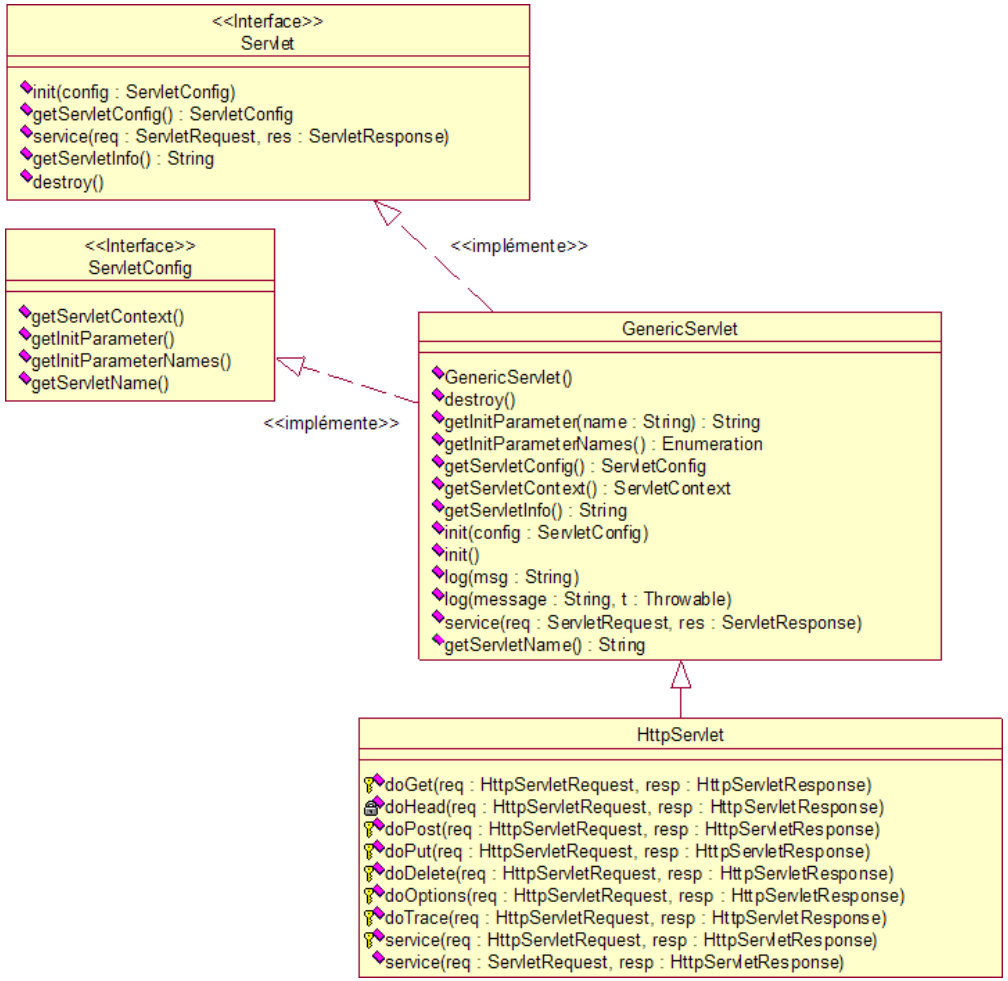
- *@WebFilter :* contrôle et restreint l'accès aux servlets
- *@WebListener :* permet d'intercepter et de réagir à des changements d'état du cycle de vie (Observer)
- *@ServletSecurity : règles de sécurité (Contraintes d'accès (@HttpConstraint), Rôles autorisés (rolesAllowed) et Sécurité des méthodes (@HttpMethodConstraint))*

- *@MultipartConfig : accepte et gère les requêtes HTTP de type multipart/form-data*

Mapping URL Servlet

Type de Mapping Exemple de Motif Description /boutique/achat		
Correspondance exacte	/boutique/achat	Déclenché uniquement si l'URL correspond au caractère près.
Préfixe avec joker	/api/*	Capture toutes les requêtes commençant par /api/ (ex: /api/users, /api/products).
Extension avec joker	*.do ou *.html	Intercepte tous les fichiers se terminant par une extension précise, peu importe le dossier.
Servlet par défaut	/	Agit comme une solution de secours pour toutes les requêtes qui ne correspondent à aucun autre mapping (y compris les ressources statiques).

Classe HttpServlet



➔Le paquetage javax.servlet contient les classes et interfaces utilisées pour écrire des servlets
➔Le paquetage javax.servlet.http contient les classes et interfaces pour écrire des servlets qui fonctionnent avec le protocole HTTP

(SOURCE https://www.reveillere.fr/M2WEB/cours/servlet.pdf)

Une servlet

```java
package iut.nantes.demo;

    import java.io.*;

    import jakarta.servlet.http.*;
    import jakarta.servlet.annotation.*;

    @WebServlet(name = "helloServlet", value = "/hello-servlet")
    public class HelloServlet extends HttpServlet {
        private String message;

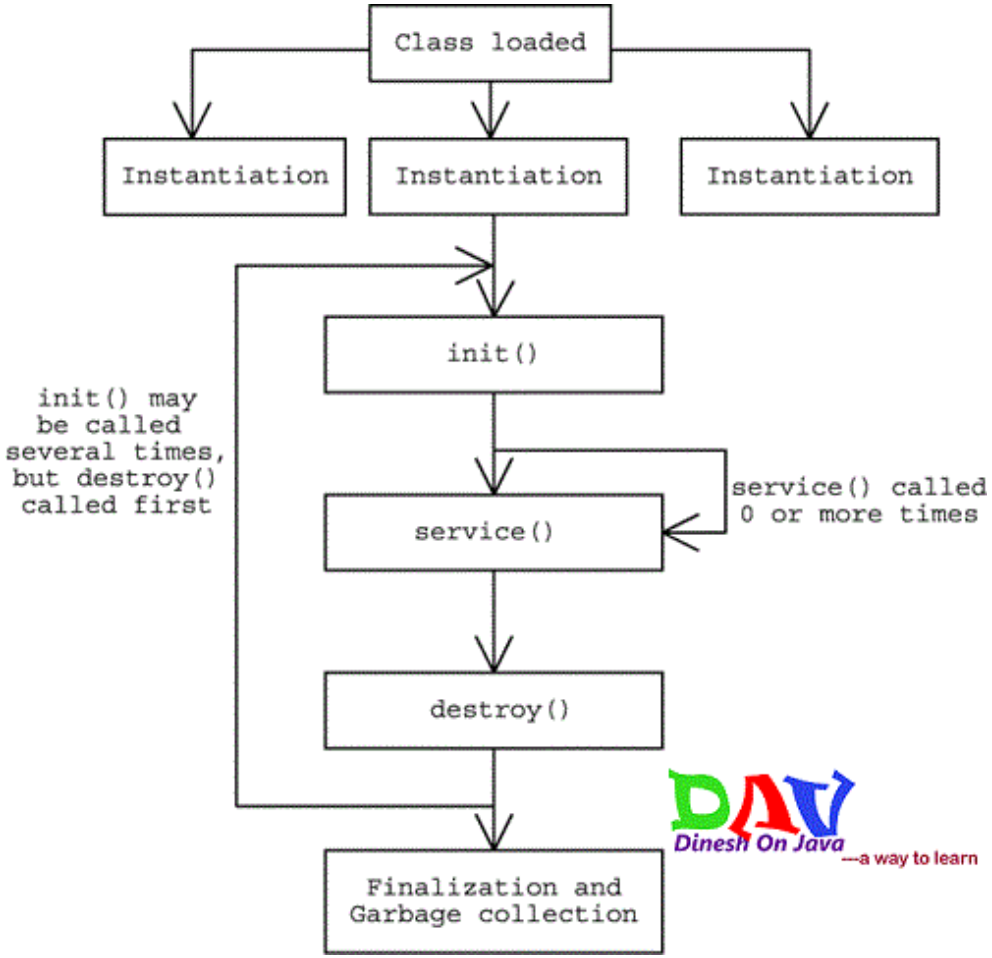
        public void init() {
            message = "Hello World";
        }

        public void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
            response.setContentType("text/html");

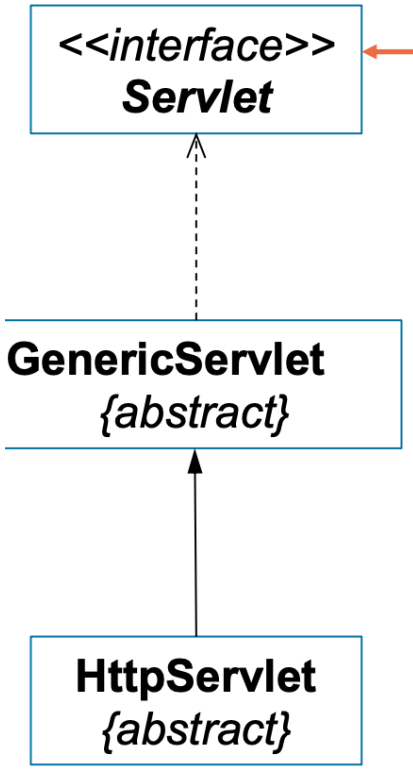
            // Hello
            PrintWriter out = response.getWriter();
            out.println("<html><body>");
            out.println("<h1>" + message + "</h1>");
            out.println("</body></html>");
        }

        public void destroy() {
            super.destroy();
        }
    }
```

Méthodes et cycle de vie



Interface Servlet



init

appelé par le conteneur à la création du servlet

service

traite une requête

destroy

appelé par le conteneur à la suppression du servlet

getServletConfig

donne accès aux paramètres d'initialisation du servlet (web.xml)

getServletInfo

informations sous forme de String

Recevoir et répondre

HttpServletRequest :

Fille de l’interface ServletRequest qui représente la requête du client au serveur

HttpServletResponse:

Fille de l'interface ServletResponse qui représente la réponse du serveur

HttpServletRequest

Cette interface encapsule toutes les informations envoyées par le client au serveur lors d'une requête HTTP. Elle permet à la servlet de lire les données entrantes.

Méthodes courantes :

- **getParameter(String name)** : Récupère la valeur d'un paramètre du formulaire ou de l'URL (Query String).
- **getParameterValues(String name)** : Retourne un tableau de chaînes (String[]) si un paramètre possède plusieurs valeurs (ex: des cases à cocher).
- **getHeader(String name)** : Récupère la valeur d'un en-tête HTTP spécifique envoyé par le navigateur (ex: User-Agent, Accept).
- **getCookies()** : Renvoie un tableau de tous les objets Cookie envoyés avec la requête.
- **getSession()** / **getSession(boolean create)** : Permet de récupérer ou de créer la session active (HttpSession) de l'utilisateur pour maintenir l'état.
- **setAttribute(String name, Object o)** / **getAttribute(String name)** : Permet de stocker et de transmettre des objets personnalisés au cours du cycle de vie de la requête (très utile lors d'un transfert vers une page JSP).
- **getMethod()** : Retourne la méthode HTTP utilisée (GET, POST, PUT, DELETE, etc.).

HttpServletResponse

Cette interface représente la réponse HTTP que le serveur va renvoyer au client. Elle permet à la servlet de configurer le contenu du message et les métadonnées de transmission.

Méthodes courantes :

- **setContentType(String type)** : Définit le type MIME du contenu renvoyé (ex: text/html pour du code HTML, application/json pour une API).
- **getWriter()** : Retourne un objet PrintWriter permettant d'écrire du texte directement dans le corps de la réponse (généralement pour générer du HTML ou du JSON).
- **getOutputStream()** : Retourne un flux binaire (ServletOutputStream), indispensable pour envoyer des fichiers lourds comme des images, des PDF ou des vidéos.
- **setStatus(int sc)** : Définit le code d'état HTTP de la réponse (ex: HttpServletResponse.SC_OK pour un succès 200, ou SC_NOT_FOUND pour une erreur 404).
- **setHeader(String name, String value)** : Configure un en-tête HTTP de réponse.
- **addCookie(Cookie cookie)** : Ajoute un cookie au navigateur du client.
- **sendRedirect(String location)** : Redirige temporairement le client vers une autre URL

Service

La méthode service de la classe HTTPServlet délègue le traitement à une autre méthode, selon le type de requête HTTP envoyée par le client :

- doGet()
- doPost()
- doPut()
- doDelete()
- doHead()
- doOptions()
- doTrace()

Thread

- Création ou réutilisation d'un thread par requête par le container de servlet
  - Pour traiter chaque connexion ultérieure qui doit être traitée par le même type de servlet
- Variables d'instance des servlets
  - Accessibles par tous les clients
  - Ne pas garder d'informations spécifiques à un client
- Éventuelles informations conservées dans le servlet
  - Protection contre les accès concurrents

Récupérer les paramètres d’initialisation

- Méthodes de l’interface ServletConfig
  - HttpServlet implémente ServletConfig
- ServletConfig contient les méthodes
  - Enumeration getInitParameterNames()
    - retourne une énumération des noms des paramètres
  - String getInitParameter(String)
    - retourne la valeur du paramètre d’initialisation dont le nom est passé en paramètre

Récupérer les paramètres d'initialisation

```
@WebServlet(
        urlPatterns={"/webservlet1","/webservlet2"},
        initParams = {
                @WebInitParam(name="valeurInit",value = "0"),
        }
  )
```

```
valeur = Integer.valueOf(getInitParameter("valeurInit"));
```

Contexte d’une application

- Plusieurs applications hébergées par le même serveur Web
  - Chaque application Web décrite par un fichier web.xml (ou par des annotations) définit un contexte
    - interface ServletContext qui représente le contexte dans lequel l’application s’exécute
- Durant l’exécution
  - URL donné par le client détermine le contexte utilisé
    - Obtenir le contexte : request.getContextPath()
    - Exemple pour « http:/host.com/appli/liste »

Forward et redirect

- Redirect on informe le client que l’on n’a pas la ressource mais que l’on sait on la trouvée

```
response.sendRedirect("http://appli/truc?n1=val1");
```

- Forward en interne, on passe la requette à une autre servlet

```
request.getRequestDispatcher("/servlet1").forward(request, response);
```

Conserver des données

- Entre les servlets de l’application
- Pour la persistance dans un formulaire
- Pendant une session
- A travers une requête(forwrd)

Conserver les données : portée

- Requête
- Cookie
- Session
- Application

Conserver les données : Requête

depuis l’arrivée de la requête jusqu’au renvoi de la réponse au client

Accès par : HttpServletRequest

Conserver les données : Cookie

jakarta.servlet.http.Cookie

- Créer et envoyer un cookie au client

```
Cookie cookie = new Cookie("login", login);

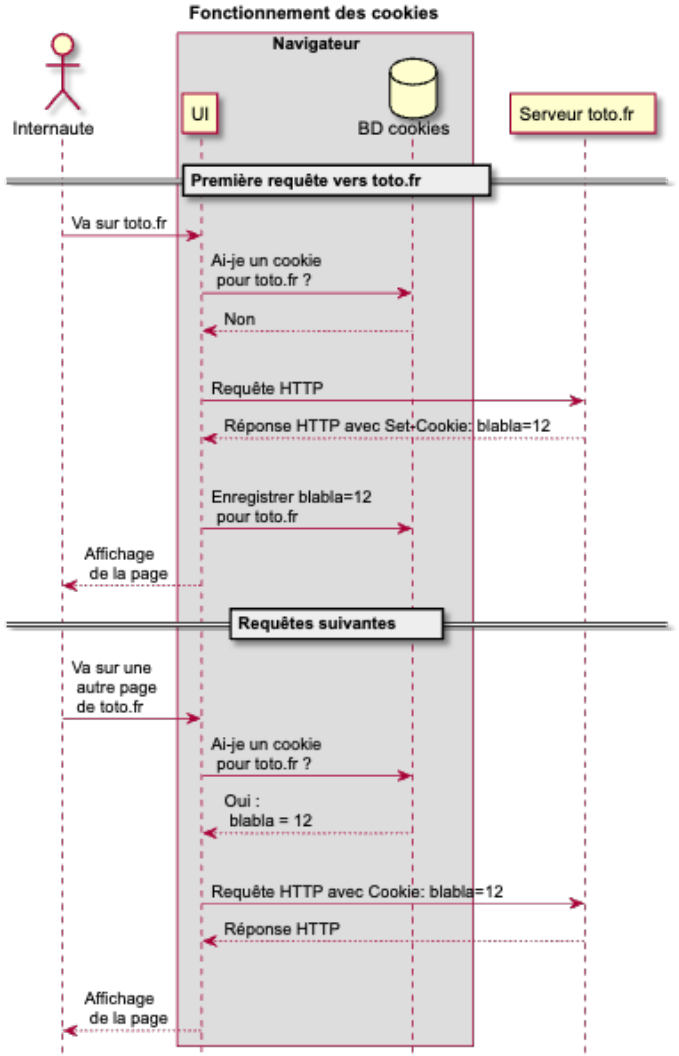
    // Durée de vie : 1 hueure
    cookie.setMaxAge(60 * 60);

    // Le cookie est disponible sur toute l'application
    cookie.setPath("/");

    response.addCookie(cookie);
```

- Lire les cookies envoyés par le client

```
Cookie[] cookies = request.getCookies();
```



Conserver les données : sessions

HttpSession session = request.getSession(true);

Retourne la session en cours, en crée une (true) si il n’en existe pas, isNew() pour tester

L’objet session fonctionne comme une HashMap

HttpSession session = request.getSession();
session.setAttribute("name", "value");

session.getAttribute("name");

session.removeAttribute("name");

Conserver les données : sessions

- Time out
  - public int getMaxInactiveInterval();
  - public void setMaxInactiveInterval (int seconds);
- Etat
  - public long getLastAccessedTime();
  - public long getCreationTime();
  - public boolean isNew();
  - public String getId();

Conserver les données : sessions

- Quand on appelle getSession(true) chaque utilisateur se voit attribuer un Session ID
  - Le Session ID est communiqué au client
    - Option 1: si le navigateur supporte les cookies, la Servlet crée un cookie avec le session ID, dans Tomcat, ce cookie est appelé JSESSIONID
    - Option 2: si le navigateur ne supporte pas les cookies, la servlet va essayer de récupérer le session ID depuis l'URL

Conserver les données : application

Dans la servlet

```
public void init(ServletConfig config) throws ServletException {
super.init(config);
// Récupération via le ServletConfig
ServletContext context = config.getServletContext();
}
```

Mais aussi

```
ServletContext context = request.getServletContext();
ServletContext context = session.getServletContext();
ServletContext context = filterConfig.getServletContext();
```

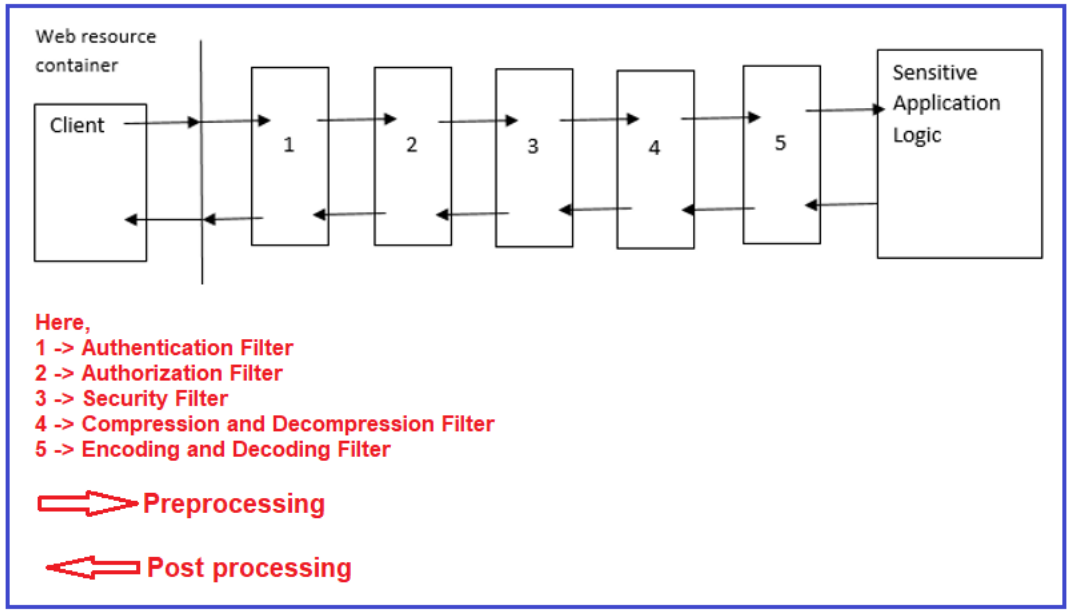
Conserver les données : application

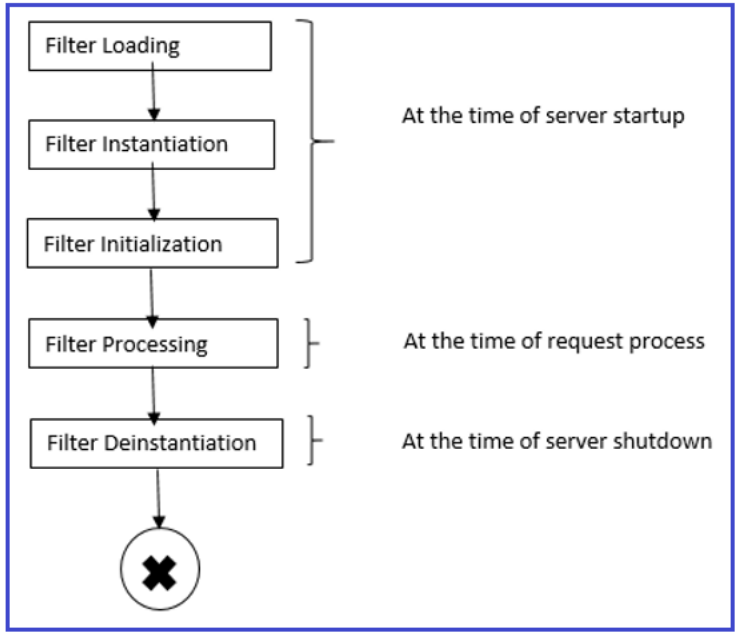
Ajouter, modifier des attributs

```
ServletContext context = getServletContext();
context.setAttribute("appTheme", "dark-mode");
context.setAttribute("totalUsersConnected", 150);

Integer count = (Integer) context.getAttribute("totalUsersConnected");
context.setAttribute("totalUsersConnected", count + 1);
```

Les filtres


Web resource container
Client
1
2
3
4
5
Sensitive Application Logic
Here,
1 -> Authentication Filter
2 -> Authorization Filter
3 -> Security Filter
4 -> Compression and Decompression Filter
5 -> Encoding and Decoding Filter
Preprocessing
Post processing


Filter Loading
Filter Instantiation
Filter Initialization
Filter Processing
Filter Deinstantiation
At the time of server startup
At the time of request process
At the time of server shutdown

Les filtres

- Objet qui intercepte
  - Les requêtes avant qu'elles ne soient traitées par un servlet
  - Les réponses juste après qu'elles aient été générées par un servlet
- Chaînage des filtres

- Annotation @WebFilter

```
@WebFilter(
filterName = "Filtre1",
urlPatterns = {"/*"},
initParams = { @WebInitParam(name="p1", value="v1") } )

public class Filtre1 implements Filter { ... }
```

Le cache du navigateur

Demander au navigateur de ne pas utiliser son cache

```
response.setHeader("Pragma", "No-cache");
response.setDateHeader("Expires", 0);
response.setHeader("Cache-Control", "no-cache");
```

Mise en ouvre

Si l'authentification fonctionne alors le login est conservé

